Machine Coding: Infinite Scroll

- **Problem:** Build an infinite scrolling list that loads the next page of posts automatically as the user nears the bottom.
- **Tech stack:** React (Vite, JavaScript), plain CSS
- **Time limit:** 60 minutes

PART A: Thought process

The same 12 steps work for any machine coding task:

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List interactions / use cases:** every action and its expected result. This is your checklist.
3. **Design the architecture:** components, hooks, services and who owns what.
4. **Design the state:** the smallest set of state; derive the rest.
5. **Design data and contracts:** example data, API shape, function signatures.
6. **Trace the data flow:** one interaction end to end before writing code.
7. **Find the hard parts:** the 2-4 things that break naive versions.
8. **Plan the build order:** smallest working version first, then one layer at a time.
9. **Cover states and edge cases:** loading, success, empty, error, end, and problem-specific ones.
10. **Fix the file structure.**
11. **Write the code.**
12. **Wrap up:** what is handled, trade-offs, what you would add next.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Use cases, architecture, state, contracts	8 min
Build: fetch + render first page	10 min
Build: sentinel + IntersectionObserver + paging	12 min
Build: loading / error / retry / end states	10 min
Hard parts: duplicate requests, StrictMode, under-filled viewport	8 min
Polish, accessibility, walkthrough	7 min

Think out loud. Say what you are deciding and why ("I'll use IntersectionObserver instead of a scroll listener because..."). The interviewer grades your reasoning as much as the result, and talking lets them steer you before you waste time.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show a list of posts (id, title, body) from a paginated API.
- Load the first page on mount.
- When the user scrolls near the bottom, load the next page and append it.
- Show a loading indicator while a page is loading.
- Show an error with a **Retry** button if a page fails; retry the same page.
- Stop requesting when there are no more items and show "You have reached the end".
- Show an empty state if the first page has no items.

Non-functional requirements

- **Performance:** no scroll event spam; at most one request in flight; prefetch slightly before the bottom so the user rarely sees a spinner.
- **Correctness:** no duplicate pages, no skipped pages, no out-of-order pages, even under React StrictMode and fast scrolling.
- **Error handling:** a failed page never advances the page counter; aborted requests never touch state.
- **Accessibility:** `role="feed"`, `aria-busy` while loading, a polite live region for status, keyboard-scrollable container, a real `<button>` for retry.
- **Reusability:** the "load more when visible" logic lives in a generic `useInfiniteScroll` hook, not inside the feed.
- **Extensibility:** swapping the API, page size or card component should not touch the scrolling logic.

Clarifying questions to ask

1. Does the list scroll inside a fixed-height container or the whole page (window)? This decides the observer `root`.
2. Page-number or cursor-based API? Does it tell us the total count?
3. Is there a "Load more" button fallback, or is it purely automatic?
4. How long can the list get? Hundreds, or tens of thousands (do we need virtualization)?
5. Should scroll position and loaded pages survive navigating away and coming back?
6. On error, auto-retry with backoff or a manual Retry button?

Assumptions we build with

- API: `https://jsonplaceholder.typicode.com/posts?_page=N&_limit=10`, 100 posts total, 10 per page.
- The list scrolls inside a fixed-height container (`80vh`), so the container is the observer root.
- A page shorter than `PAGE_SIZE` means it is the last page.
- Manual Retry on error; no virtualization (100 items is small).
- No persistence across navigation.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page opens	"Loading posts…" shows, page 1 (posts 1-10) renders
User scrolls to ~200px from the bottom	Page 2 request starts, posts 11-20 append below
User scrolls very fast / flings to the bottom	Exactly one request for the next page, never two
Page loads but content is shorter than the container	Next page loads automatically until the container is filled
Request fails (offline, 500)	Error text + Retry button; page counter does not move
User clicks Retry	Same page is requested again; on success the list continues
Last page returns fewer than 10 items (or 0)	No more requests; "You have reached the end"
First page returns 0 items	"No posts yet."
Component unmounts mid-request	Request aborted; no state update on an unmounted component
React StrictMode double-mounts in dev	First request aborted, second one used; no duplicate posts
Keyboard user tabs to the list	Container is focusable and scrolls with arrow keys / Page Down

This table is the checklist for both implementation and testing.

3. Design the architecture

Module	Responsibility
`api/posts.js`	`fetchPosts(page, signal)` : one HTTP call, throws on non-2xx
`utils/mergeUnique.js`	Pure function: append a page skipping ids already present
`hooks/useInfiniteScroll.js`	Generic: watches a sentinel element, calls `onLoadMore` once when it becomes visible
`components/PostFeed.jsx`	Owns the list state, fetches pages, renders cards + status + sentinel
`components/PostCard.jsx`	Presentational, memoized card for one post
`App.jsx`	Page shell (heading + feed)

```
App
└── PostFeed                (state: posts, page, status, error, hasMore, attempt)
    ├── uses fetchPosts()   (api/posts.js)
    ├── uses mergeUnique()  (utils)
    ├── uses useInfiniteScroll({ rootRef, enabled, onLoadMore }) -> sentinelRef
    ├── PostCard × N
    ├── status region       (loading / error + Retry / empty / end)
    └── <div ref={sentinelRef} />   (invisible 1px trigger at the end)
```

Start simple: the first version can be one component. Split the hook out once the observer logic works, because that is the reusable part.

4. Design the state / data model

State or field	Purpose
`posts`	All loaded posts, in order
`page`	The page that should be loaded now; changing it triggers a fetch
`status`	`'loading' | 'idle' | 'error'`: one field instead of separate booleans that can contradict
`error`	Error message for the error state
`hasMore`	`false` once a short page arrives; stops the observer
`attempt`	Retry counter; bumping it re-runs the fetch for the same `page`
`containerRef`	DOM ref to the scroll container (observer root)
`sentinelRef`	DOM ref to the trigger element (returned by the hook)

```
const [posts, setPosts] = useState([]);
const [page, setPage] = useState(1);
const [status, setStatus] = useState('loading'); // 'loading' | 'idle' | 'error'
const [error, setError] = useState(null);
const [hasMore, setHasMore] = useState(true);
const [attempt, setAttempt] = useState(0);
const containerRef = useRef(null);
```

Senior notes

- **Derived, not stored:** `isEmpty = status === 'idle' && posts.length === 0`, `canLoadMore = status === 'idle' && hasMore`. Storing them would mean keeping them in sync by hand.
- **`status` starts as `'loading'`,** not `'idle'`. If it started idle, the observer would arm on the first render, see the sentinel (the list is empty, so it is visible) and request page 2 before page 1 arrives.
- **Ownership:** `PostFeed` owns all list state. The hook owns nothing but the observer; it gets `enabled` and `onLoadMore` from outside, so it works for any list.
- **Why `page` is state and not a ref:** the fetch is a reaction to the page changing. Putting `page` in the effect's dependency array gives us cancellation (cleanup aborts the old request) for free.

5. Design the data structures and contracts

Example data (each post has a stable numeric `id`, used as the React `key` and for de-duplication):

```
[
  { "userId": 1, "id": 11, "title": "et ea vero quia laudantium autem", "body": "delectus reiciendis molestiae…" },
  { "userId": 1, "id": 12, "title": "in quibusdam tempore odit est dolorem", "body": "itaque id aut magnam…" }
]
```

Stable ids matter: an index key would reuse DOM nodes wrongly if a page were ever inserted or de-duplicated, and de-duplication needs something unique to compare.

API contract

```
GET https://jsonplaceholder.typicode.com/posts?_page={page}&_limit=10
200 -> Post[]   (10 items; fewer on the last page; [] past the end)
Header: x-total-count: 100   (available, not required by our design)
```

Function / hook contracts

```
fetchPosts(page: number, signal: AbortSignal) -> Promise<Post[]>   // throws on !res.ok or abort
mergeUnique(prev: Post[], next: Post[]) -> Post[]                   // returns prev itself if nothing new
useInfiniteScroll({
  rootRef,        // ref to scroll container, or omit for the window
  enabled,        // boolean: arm the observer only when a load is allowed
  onLoadMore,     // () => void, called at most once per arm
  rootMargin,     // default '200px': prefetch distance
}) -> sentinelRef
```

6. Trace the data flow

User scrolls down near the end of page 2:

```
scroll inside .feed
  ↓ sentinel enters root + 200px margin
IntersectionObserver callback (entry.isIntersecting = true)
  ↓ observer.disconnect()          // fire once only
  ↓ onLoadMore() -> setPage(p => p + 1)        page 2 -> 3
render, then fetch effect runs for page 3
  ↓ cleanup of the previous effect run (nothing in flight)
  ↓ setStatus('loading')  -> enabled = false, hook cleans up, aria-busy = true, "Loading posts…"
  ↓ fetchPosts(3, controller.signal)
response 200 with 10 posts
  ↓ setPosts(prev => mergeUnique(prev, data))  posts 1-30
  ↓ setHasMore(10 === PAGE_SIZE) -> true
  ↓ setStatus('idle') -> enabled = true
render: 30 cards; hook effect re-arms a new observer
  ↓ observer.observe(sentinel) reports the current state immediately
  ↓ if the sentinel is still within 200px (screen not filled), load page 4; otherwise wait for
scrolling
```

7. Important technical challenges

Challenge 1: Detecting "near the bottom" without scroll-event spam

- - **Problem:** A `scroll` listener fires dozens of times per second. Each call reads `scrollTop + clientHeight >= scrollHeight - 200`, forcing layout reads on the main thread; without throttling it causes jank, and with throttling it can miss the bottom on a fast fling.
- **Solution:** Put a 1px **sentinel** element after the last card and let `IntersectionObserver` tell us when it comes within `rootMargin: '200px'` of the scroll container. The browser computes intersections off the hot path; we get one callback per visibility change.

```
const observer = new IntersectionObserver(
  ([entry]) => { if (entry.isIntersecting) onLoadMore(); },
  { root: containerRef.current, rootMargin: '200px' }
);
observer.observe(sentinelRef.current);
```

Note `root`: if the list scrolls inside a container, pass the container. With `root: null` the margin applies to the viewport, not to the box that actually scrolls.

Challenge 2: Duplicate and out-of-order requests

- - **Problem:** Scenario: the sentinel is visible, the callback fires, `setPage(3)` runs, but before React re-renders and disables the observer, a second intersection callback fires and calls `setPage(4)`. Now pages 3 and 4 load together, or page 3 is skipped. In dev, StrictMode mounts, unmounts and re-mounts, so page 1 is fetched twice and appended twice (duplicate keys warning).
- **Solution:** Three layers, each cheap:
  1. The hook **disconnects inside the callback** before calling `onLoadMore`, so one arm = one call.

2. The observer is only armed when `status === 'idle' && hasMore`; while loading there is no observer at all.
3. The fetch effect returns `controller.abort()`. StrictMode's first run is aborted, and the `AbortError` is ignored. `mergeUnique` is a final safety net if a page is ever applied twice.

```
useEffect(() => {
  const controller = new AbortController();
  fetchPosts(page, controller.signal)
    .then((data) => setPosts((prev) => mergeUnique(prev, data)))
    .catch((err) => { if (err.name !== 'AbortError') setStatus('error'); });
  return () => controller.abort();
}, [page, attempt]);
```

Challenge 3: The screen is not filled after the first page

- - **Problem:** On a tall monitor, 10 cards may not fill the container. The sentinel stays visible the whole time, so there is no "enters view" change and an observer that was created once never fires again. The user sees 10 posts, no scrollbar and nothing more.
- **Solution:** Re-create the observer each time loading finishes (the hook's effect depends on `enabled`). A freshly observed element is reported immediately with its current state, so if the sentinel is still visible the next page loads, repeatedly, until the container is full or the data ends.

```
useEffect(() => {
  if (!enabled || !node) return;
  const observer = new IntersectionObserver(cb, opts);
  observer.observe(node);      // initial callback = "is it visible right now?"
  return () => observer.disconnect();
}, [enabled, onLoadMore, rootRef, rootMargin]);
```

`onLoadMore` is wrapped in `useCallback` in the parent so it does not re-create the observer on every render.

Challenge 4: Errors, retry and knowing when to stop

- - **Problem:** If the page counter advances before the request succeeds, a failure loses that page forever. If we never detect the end, the observer keeps requesting empty pages.
- **Solution:** `page` changes only from the observer; a failure keeps `page` as is and sets `status = 'error'` (observer disabled). Retry bumps `attempt`, which is in the effect's dependencies, so the same page is fetched again. The end is detected when a page returns fewer than `PAGE_SIZE` items.

```
const retry = () => setAttempt((a) => a + 1);       // same page, new request
setHasMore(data.length === PAGE_SIZE);              // short page = last page
```

Trade-off: if the total is an exact multiple of 10 (it is: 100), one extra request returns `[]` before we stop. Using the `x-total-count` header would avoid it, at the cost of coupling to that header.

8. Implement in the correct order

#	Step	Working result
1	Static `PostCard` + `App` with 3 hard-coded posts	Layout and styling done
2	`fetchPosts` service + fetch page 1 in `useEffect`	Real first page renders
3	Add `status` (`loading` / `idle` / `error`) and the status region	Loading and error visible
4	Add `page` state, fetch effect depends on `[page]`, append results	Can load page 2 with a temporary "Load more" button
5	Add the sentinel + `IntersectionObserver` with the container as root	Scrolling loads pages automatically
6	Gate the observer on `status === 'idle' && hasMore`, disconnect in the callback	No double requests
7	`AbortController` cleanup + ignore `AbortError`	StrictMode and unmount safe
8	`hasMore` from short page + end message + empty state	Stops at the end
9	`attempt` counter + Retry button	Failed page can be retried
10	Extract `useInfiniteScroll` hook, `useCallback` for `onLoadMore`	Reusable, observer stable
11	`mergeUnique`, `memo(PostCard)`	Duplicate-proof, fewer re-renders
12	Accessibility: `role="feed"`, `aria-busy`, live region, `tabIndex`	Screen reader and keyboard friendly

9. Cover all states and edge cases

State / edge case	What the user sees	How it is handled
Initial	"Loading posts…"	`status` starts as `'loading'`
Loading next page	Existing posts + "Loading posts…" at the bottom	Observer disabled; `aria-busy="true"`
Success	New posts appended	`mergeUnique`, `status = 'idle'`, observer re-armed
Empty	"No posts yet."	`status === 'idle' && posts.length === 0`
Error (first page or later)	Message + Retry; loaded posts stay	`status = 'error'`; `page` unchanged
Retry	Loading, then the same page	`attempt + 1` re-runs the effect
End of data	"You have reached the end."	`hasMore = false`; observer never armed again
Viewport not filled	More pages load until it is	Observer re-created after each load reports current visibility
Fast fling to the bottom	One request at a time, in order	Disconnect in callback + gated `enabled`

State / edge case	What the user sees	How it is handled
StrictMode double mount	One set of posts	First request aborted, `AbortError` ignored
Unmount mid-request	Nothing	Cleanup aborts the request
Same post returned twice	Shown once	`mergeUnique` by `id`
Slow network	Prefetch hides most of it	`rootMargin: '200px'` starts early
Keyboard user	Can focus and scroll the list	`tabIndex={0}` + focus outline

10. Final structure

```
_19_infinite_scroll_bar/
└── src/
    ├── api/
    │   └── posts.js              // fetchPosts, PAGE_SIZE
    ├── utils/
    │   └── mergeUnique.js        // pure de-dup append
    ├── hooks/
    │   └── useInfiniteScroll.js  // sentinel + IntersectionObserver
    ├── components/
    │   ├── PostFeed.jsx          // state, fetching, states UI
    │   └── PostCard.jsx          // one post
    ├── App.jsx
    ├── App.css
    ├── index.css
    └── main.jsx
```

11. Full working code

`src/api/posts.js`

```
const BASE_URL = 'https://jsonplaceholder.typicode.com/posts';
export const PAGE_SIZE = 10;

// GET /posts?_page=2&_limit=10 -> [{ userId, id, title, body }, ...]
export async function fetchPosts(page, signal) {
  const res = await fetch(`${BASE_URL}?_page=${page}&_limit=${PAGE_SIZE}`, { signal });
  if (!res.ok) throw new Error(`Request failed (${res.status})`);
  return res.json();
}
```

src/utils/mergeUnique.js

```
// Append the new page, skipping ids we already have (retries, shifted pages).
export function mergeUnique(prev, next) {
  const seen = new Set(prev.map((item) => item.id));
  const fresh = next.filter((item) => !seen.has(item.id));
  return fresh.length ? [...prev, ...fresh] : prev;
}
```

src/hooks/useInfiniteScroll.js

```
import { useEffect, useRef } from 'react';

// Calls onLoadMore once when the sentinel scrolls into view (rootMargin early).
// Re-arms whenever `enabled` flips back to true, e.g. after a page finishes loading.
export function useInfiniteScroll({ rootRef, enabled, onLoadMore, rootMargin = '200px' }) {
  const sentinelRef = useRef(null);

  useEffect(() => {
    const node = sentinelRef.current;
    if (!enabled || !node) return;

    const observer = new IntersectionObserver(
      ([entry]) => {
        if (!entry.isIntersecting) return;
        observer.disconnect(); // fire once per arm, never twice for one page
        onLoadMore();
      },
      { root: rootRef?.current ?? null, rootMargin }
    );
    observer.observe(node); // observe() reports the current state right away
    return () => observer.disconnect();
  }, [enabled, onLoadMore, rootRef, rootMargin]);

  return sentinelRef;
}
```

`src/components/PostCard.jsx`

```
import { memo } from 'react';

function PostCard({ post, index, total }) {
  return (
    <article className="card" aria-posinset={index + 1} aria-setsize={total}>
      <span className="card-id">{post.id}</span>
      <h3 className="card-title">{post.title}</h3>
      <p className="card-body">{post.body}</p>
    </article>
  );
}

export default memo(PostCard);
```

`src/components/PostFeed.jsx`

```
import { useCallback, useEffect, useRef, useState } from 'react';
import { fetchPosts, PAGE_SIZE } from '../api/posts';
import { mergeUnique } from '../utils/mergeUnique';
import { useInfiniteScroll } from '../hooks/useInfiniteScroll';
import PostCard from './PostCard';

export default function PostFeed() {
  const [posts, setPosts] = useState([]);
  const [page, setPage] = useState(1);
  const [status, setStatus] = useState('loading'); // 'loading' | 'idle' | 'error'
  const [error, setError] = useState(null);
  const [hasMore, setHasMore] = useState(true);
  const [attempt, setAttempt] = useState(0); // bump to retry the same page
  const containerRef = useRef(null);

  // Fetch whenever the page (or a retry) changes; abort if it changes again first.
  useEffect(() => {
    const controller = new AbortController();
    setStatus('loading');
    setError(null);

    fetchPosts(page, controller.signal)
      .then((data) => {
        setPosts((prev) => mergeUnique(prev, data));
        setHasMore(data.length === PAGE_SIZE);
        setStatus('idle');
      })
      .catch((err) => {
        if (err.name === 'AbortError') return;
        setError(err.message);
        setStatus('error');
      });

    return () => controller.abort();
  }, [page, attempt]);

  const loadMore = useCallback(() => setPage((p) => p + 1), []);
  const retry = () => setAttempt((a) => a + 1);

  const sentinelRef = useInfiniteScroll({
    rootRef: containerRef,
    enabled: status === 'idle' && hasMore,
    onLoadMore: loadMore,
  });
```

```
  const isEmpty = status === 'idle' && posts.length === 0;

  return (
    <section
      ref={containerRef}
      className="feed"
      role="feed"
      aria-busy={status === 'loading'}
      aria-label="Posts"
      tabIndex={0}
    >
      {posts.map((post, i) => (
        <PostCard key={post.id} post={post} index={i} total={hasMore ? -1 : posts.length} />
      ))}

      <div className="feed-status" role="status" aria-live="polite">
        {status === 'loading' && <p className="loader">Loading posts…</p>}
        {status === 'error' && (
          <div className="error">
            <p>Could not load posts: {error}</p>
            <button type="button" onClick={retry}>Retry</button>
          </div>
        )}
        {isEmpty && <p>No posts yet.</p>}
        {!hasMore && posts.length > 0 && <p>You have reached the end.</p>}
      </div>

      <div ref={sentinelRef} className="sentinel" aria-hidden="true" />
    </section>
  );
}
```

src/App.jsx

```
import PostFeed from './components/PostFeed';
import './App.css';

export default function App() {
  return (
    <main>
      <h1>Infinite Scroll</h1>
      <PostFeed />
    </main>
  );
}
```

`src/App.css`

```
h1 {
  text-align: center;
}

.feed {
  width: min(420px, 100%);
  height: 80vh;
  margin: 32px auto 0;
  padding: 16px;
  overflow-y: auto;
  display: flex;
  flex-direction: column;
  gap: 16px;
  border: 1px solid #ddd;
  border-radius: 8px;
}

.feed:focus-visible {
  outline: 2px solid #3991ef;
}

.card {
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 6px;
  background: wheat;
}

.card-id {
  display: inline-grid;
  place-items: center;
  width: 30px;
  height: 30px;
  margin-bottom: 6px;
  border-radius: 50%;
  background: #3991ef;
  color: #fff;
}

.card-title {
  font-size: 18px;
}

.card-body {
  display: -webkit-box;
```

```
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

.feed-status {
  text-align: center;
}

.error button {
  margin-top: 8px;
  padding: 4px 12px;
}

.sentinel {
  height: 1px;
  flex-shrink: 0;
}
```

`main.jsx` and `index.css` stay as the Vite template creates them (`<StrictMode><App /></StrictMode>`).

12. Wrap-up

Edge cases handled: first load, loading more, empty, error with retry, end of data, under-filled viewport, fast scrolling, StrictMode double mount, unmount mid-request, duplicate items, keyboard access.

Trade-offs to mention

- **IntersectionObserver vs scroll listener:** less code and no jank; needs the right `root` . Old browsers (pre-2019) would need a polyfill.
- `page` **in state + effect vs an imperative** `loadMore()` **with a** `loadingRef` **guard:** the effect gives cancellation and retry for free and keeps one source of truth; the cost is that the fetch is one render later than the click/intersection.
- **Short page = end:** simple and API-agnostic, but costs one empty request when the total is a multiple of the page size.
- **No virtualization:** fine for hundreds of items; thousands of DOM nodes will slow scrolling and memory.
- `status` **enum vs booleans:** impossible states (`loading && error`) cannot happen.

With more time

- Tests: `mergeUnique` unit tests; React Testing Library with a mocked `IntersectionObserver` and `fetch` for "loads page 2 on intersect", "retry refetches the same page", "stops at the end".
- Virtualize with `react-window` / `@tanstack/react-virtual` once the list gets long.
- Cache pages and restore the scroll position on back navigation (or use TanStack Query's `useInfiniteQuery`).
- Skeleton cards instead of a text loader; auto-retry with exponential backoff.
- Cursor-based pagination support for feeds where items are inserted at the top.

13. Counter questions the interviewer may ask

Requirements

Q1. Why infinite scroll and not pagination with page numbers?

Infinite scroll suits browsing feeds where the user has no specific target. Pagination is better when users need to reach the footer, bookmark a position, or jump to "page 7". I would ask the product goal; a "Load more" button is a middle ground that keeps the footer reachable.

Q2. The page itself scrolls, not a container. What changes?

Only the observer root: pass `rootRef` as `undefined` so `root` is `null` (the viewport), and remove the fixed height / `overflow` from `.feed`. The hook already supports that.

Q3. How do you know there is no more data?

A page with fewer than `PAGE_SIZE` items. Alternatives: the `x-total-count` header (`posts.length >= total`), a `hasNextPage` flag, or a `nextCursor: null` from a cursor API. I would use whatever the API gives explicitly and fall back to the short-page rule.

State design

Q4. Why a `status` string instead of `isLoading` and `isError` booleans?

Two booleans allow four combinations, one of which (`loading && error`) is meaningless and causes bugs. One enum allows exactly the valid states and makes the UI a simple switch on `status`.

Q5. Why is `page` state and not derived from `posts.length / PAGE_SIZE`?

`page` is the *requested* page, not the loaded one. During loading and after an error the two differ, and with de-duplication `posts.length` may not be a multiple of 10. Storing the intent keeps the effect simple: "whenever the requested page changes, fetch it".

Q6. Why does `status` start as `'loading'`?

Because the first fetch starts on mount. If it started as `'idle'`, the observer would arm on the first render, see the visible sentinel in the empty list and call `setPage(2)` before page 1 arrives.

Hard part: detecting the bottom

Q7. Why not a scroll event listener?

It fires on every scroll frame and needs layout reads (`scrollHeight`, `scrollTop`) each time, so it needs throttling, and throttling can miss a fast fling. IntersectionObserver gives one callback when visibility changes, computed by the browser.

Q8. What does `rootMargin: '200px'` do and how would you pick the value?

It grows the root box by 200px, so the sentinel "intersects" 200px before it is actually visible: the next page starts loading early. Pick roughly one or two card heights, or more on slow networks.

Q9. Why disconnect inside the callback?

Between `setPage` and React's re-render (which disables the observer), another callback can fire. Disconnecting first guarantees one `onLoadMore` per arm, so the same page is never requested twice.

Hard part: requests

Q10. What happens in React StrictMode in development?

Effects run, clean up and run again. The first page-1 request is aborted by the cleanup and its `AbortError` is ignored, so only the second request updates state. Without the abort you would see posts 1-10 twice.

Q11. What if responses arrive out of order (page 3 before page 2)?

They cannot here: there is only ever one request in flight, because the observer is disabled while `status === 'loading'`, and a newer effect run aborts the older request. If we allowed parallel prefetching we would store pages in a map by page number and render them in order.

Q12. What if the API is slow (3-5 seconds)?

The prefetch margin hides part of it; increase `rootMargin`, show skeleton cards, and consider prefetching page N+1 as soon as page N arrives. Add a timeout with `AbortSignal.timeout(10000)` so a hung request turns into a retryable error.

Q13. What if a request fails three times in a row?

Manual Retry handles it today. For auto-retry I would add exponential backoff (1s, 2s, 4s) with a max attempt count, then fall back to the manual button; and never auto-retry 4xx errors.

Implementation steps

Q14. Why `useCallback` for `onLoadMore`?

It is a dependency of the hook's effect. A new function every render would disconnect and re-create the observer on every render. `useCallback(() => setPage(p => p + 1), [])` is stable because it uses the functional updater.

Q15. Why does the observer get re-created after every load? Is that wasteful?

It is cheap (one object), and it is what fixes the "viewport not filled" case: `observe()` reports the current visibility immediately, so a still-visible sentinel loads the next page. A single long-lived observer would never fire again in that case.

Q16. Why `memo(PostCard)`?

When page 5 appends, the existing 40 cards get the same `post` object references, so `memo` skips re-rendering them. Without it, each new page re-renders every card, which grows linearly with the list.

Performance and scale

Q17. What if the list grows to 1 million items?

Never keep a million DOM nodes. Use virtualization (`@tanstack/react-virtual` / `react-window`) so only the ~20 visible rows exist in the DOM, keep the data in a normalized cache, and consider dropping far-away pages from memory and re-fetching them when the user scrolls back.

Q18. Cards have different heights. Does that break anything?

Not this design: the sentinel is after the last card, whatever the heights. It matters only once you virtualize, where you need measured or estimated row heights.

Q19. How do you restore the scroll position when the user goes to a post and comes back?

Keep the loaded pages in a cache outside the component (context, store or TanStack Query) and save `scrollTop` on unmount (e.g. in `sessionStorage`), then restore it in a `useLayoutEffect` after the cached pages render.

Trade-offs and "why not X"

Q20. Why not use a library like `react-infinite-scroll-component` or TanStack Query?

In an interview the point is to show the mechanics. In production, `useInfiniteQuery` would handle caching, retries, deduplication and background refetch; I would keep my `useInfiniteScroll` hook for the trigger and let the library own the data.

Q21. Why `id` as the key and not the index?

Keys must identify the item, not its position. With index keys, any insert, removal or de-duplication shifts keys and React reuses the wrong DOM and state for the wrong post.

Q22. How would you make this accessible?

`role="feed"` with `aria-busy` while loading, `<article>` per item with `aria-posinset` / `aria-setsize` (`-1` while the total is unknown), a polite live region announcing loading/errors/end, a focusable container, and a real Retry `<button>`. Also provide a way to reach content after the feed (skip link or a "Load more" mode), since infinite scroll can trap keyboard users before the footer.

Q23. How would you test the IntersectionObserver logic?

Mock `IntersectionObserver` in Jest/Vitest to capture the callback, then call it with `{ isIntersecting: true }` and assert `fetch` was called with `_page=2` . Test that a second call while loading does nothing, that Retry refetches the same page, and that a short page shows the end message.